

#1

```
import pandas as pd
```

```
# Load the dataset
```

```
df = pd.read_csv("/content/WA_Fn-UseC_-HR-Employee-Attrition-selected-columns.csv")
```

```
# Number of rows and columns
```

```
print("----- Dataset Shape -----")
```

```
print("Number of Rows   :", df.shape[0])
```

```
print("Number of Columns :", df.shape[1])
```

```
# Data types
```

```
print("\n----- Data Types -----")
```

```
print(df.dtypes)
```

```
# Missing values
```

```
print("\n----- Missing Values -----")
```

```
print(df.isnull().sum())
```

```
# Statistical summary
```

```
print("\n----- Statistical Summary -----")
```

```
print(df.describe(include='all'))
```

```
----- Dataset Shape -----
```

```
Number of Rows   : 124
```

```
Number of Columns : 10
```

```
----- Data Types -----
```

OUTPUT:

```
Age          float64
Attrition     float64
BusinessTravel float64
DailyRate     float64
Department    float64
DistanceFromHome float64
Education     float64
EducationField float64
EmployeeCount float64
EmployeeNumber float64
dtype: object
```

----- Missing Values -----

```
Age          124
Attrition     124
BusinessTravel 124
DailyRate     124
Department    124
DistanceFromHome 124
Education     124
EducationField 124
EmployeeCount 124
EmployeeNumber 124
dtype: int64
```

----- Statistical Summary -----

	Age	Attrition	BusinessTravel	DailyRate	Department \
count	0.0	0.0	0.0	0.0	0.0
mean	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN

	DistanceFromHome	Education	EducationField	EmployeeCount \
count	0.0	0.0	0.0	0.0
mean	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN

max	NaN	NaN	NaN	NaN
-----	-----	-----	-----	-----

	EmployeeNumber
count	0.0
mean	NaN
std	NaN
min	NaN
25%	NaN
50%	NaN
75%	NaN
max	NaN

```
#2 Import Libraries
import pandas as pd
import matplotlib.pyplot as plt

# Read Dataset
df = pd.read_csv("/content/StudentsPerformance.csv")

# Display first 5 rows
print("First 5 Records:")
print(df.head())

# Display column names
print("\nColumn Names:")
print(df.columns)

# Create Grade Column
def get_grade(score):
    if score >= 90:
        return 'A'
    elif score >= 80:
        return 'B'
    elif score >= 70:
        return 'C'
    elif score >= 60:
        return 'D'
    else:
        return 'F'

df['Grade'] = df['math score'].apply(get_grade)

# Frequency Distribution
```

```

grade_count = df['Grade'].value_counts().sort_index()

print("\nFrequency Distribution of Grades:")
print(grade_count)

# Plot Bar Chart
plt.figure(figsize=(8,5))

plt.bar(
    grade_count.index,
    grade_count.values,
    color='skyblue',
    edgecolor='black',
    width=0.6
)

plt.title("Frequency Distribution of Students' Grades", fontsize=14)
plt.xlabel("Grades", fontsize=12)
plt.ylabel("Number of Students", fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.5)

plt.show()

```

First 5 Records:

	gender	race/ethnicity	parental level of education	lunch \
0	female	group B	bachelor's degree	standard
1	female	group C	some college	standard
2	female	group B	master's degree	standard
3	male	group A	associate's degree	free/reduced
4	male	group C	some college	standard

	test preparation course	math score	reading score	writing score
0	none	72	72	74
1	completed	69	90	88
2	none	90	95	93
3	none	47	57	44
4	none	76	78	75

Column Names:

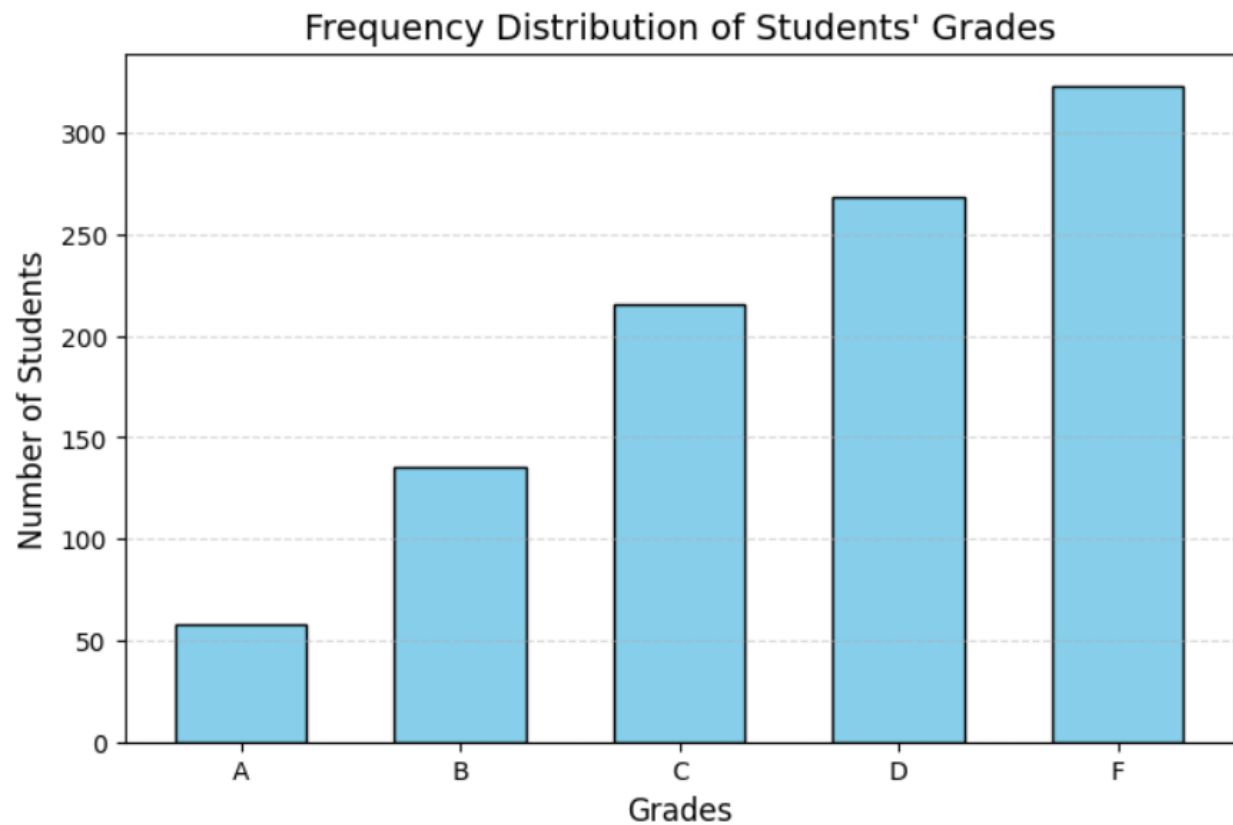
```

Index(['gender', 'race/ethnicity', 'parental level of education', 'lunch',
      'test preparation course', 'math score', 'reading score',
      'writing score'],
      dtype='object')

```

Frequency Distribution of Grades:
Grade

A 58
B 135
C 216
D 268
F 323
Name: count, dtype: int64



#3 Upload Dataset

```
from google.colab import files  
uploaded = files.upload()
```

```
# Import Libraries  
import pandas as pd  
import matplotlib.pyplot as plt
```

```
# Read Dataset  
df = pd.read_csv("/content/train.csv")
```

```
# Display first 5 rows  
print("First 5 Records:")  
print(df.head())
```

```

# Display column names
print("\nColumns:")
print(df.columns)

# Histogram
plt.figure(figsize=(8,5))
plt.hist(df["SalePrice"],
        bins=30,
        color="skyblue",
        edgecolor="black")

plt.title("Histogram of House Prices")
plt.xlabel("House Price")
plt.ylabel("Frequency")
plt.grid(axis="y", linestyle="--", alpha=0.5)

plt.show()

# Check skewness
skewness = df["SalePrice"].skew()

print("\nSkewness:", round(skewness,2))

if skewness > 0:
    print("Conclusion: The data is Positively Skewed (Right Skewed).")
elif skewness < 0:
    print("Conclusion: The data is Negatively Skewed (Left Skewed).")
else:
    print("Conclusion: The data is Normally Distributed.")

```

OUTPUT:

First 5 Records:

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	\
0	1	60	RL	65.0	8450	Pave	NaN	Reg	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	

	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	\
0	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	
1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	5	
2	Lvl	AllPub	...	0	NaN	NaN	NaN	0	9	
3	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	

4 Lvl AllPub ... 0 NaN NaN NaN 0 12

	YrSold	SaleType	SaleCondition	SalePrice
0	2008	WD	Normal	208500
1	2007	WD	Normal	181500
2	2008	WD	Normal	223500
3	2006	WD	Abnorml	140000
4	2008	WD	Normal	250000

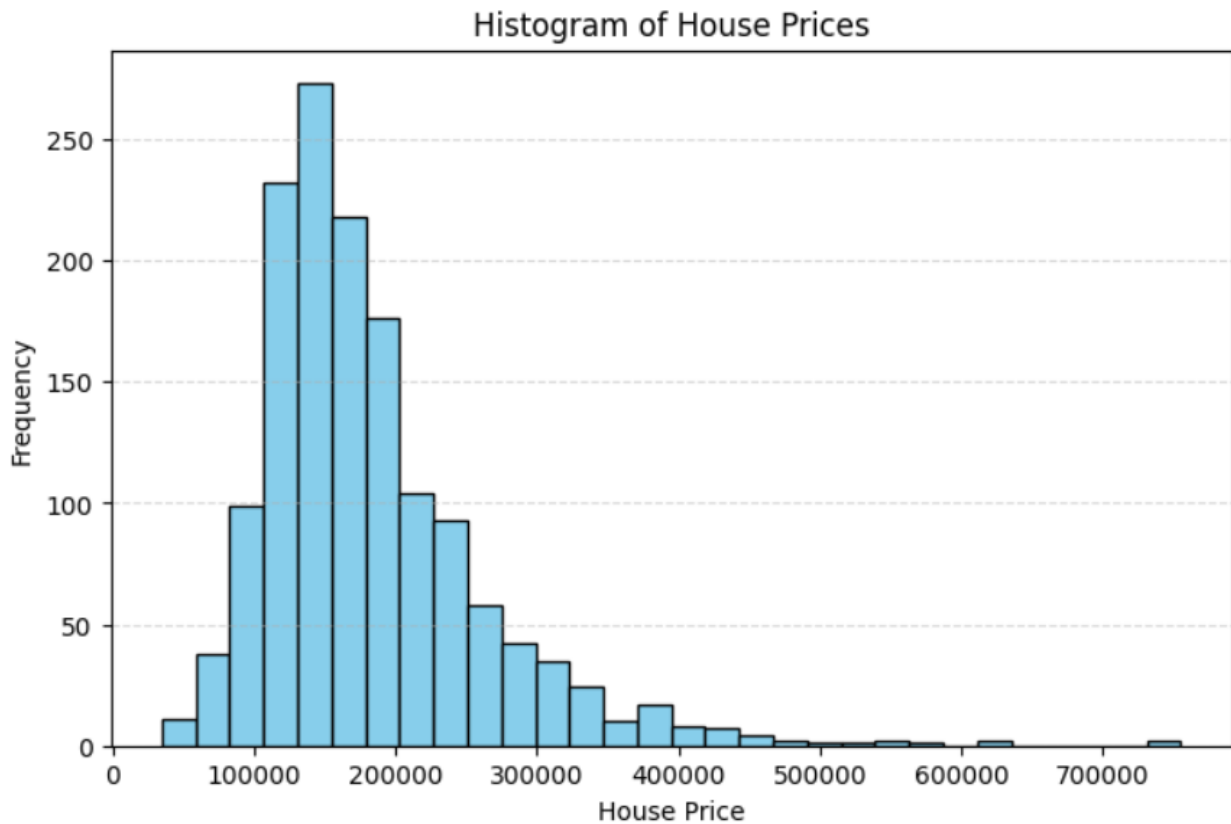
[5 rows x 81 columns]

Columns:

```
Index(['Id', 'MSSubClass', 'MSZoning', 'LotFrontage', 'LotArea', 'Street',
      'Alley', 'LotShape', 'LandContour', 'Utilities', 'LotConfig',
      'LandSlope', 'Neighborhood', 'Condition1', 'Condition2', 'BldgType',
      'HouseStyle', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd',
      'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType',
      'MasVnrArea', 'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual',
      'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinSF1',
      'BsmtFinType2', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', 'Heating',
      'HeatingQC', 'CentralAir', 'Electrical', '1stFlrSF', '2ndFlrSF',
      'LowQualFinSF', 'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath',
      'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr', 'KitchenQual',
      'TotRmsAbvGrd', 'Functional', 'Fireplaces', 'FireplaceQu', 'GarageType',
      'GarageYrBlt', 'GarageFinish', 'GarageCars', 'GarageArea', 'GarageQual',
      'GarageCond', 'PavedDrive', 'WoodDeckSF', 'OpenPorchSF',
      'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'PoolQC',
      'Fence', 'MiscFeature', 'MiscVal', 'MoSold', 'YrSold', 'SaleType',
      'SaleCondition', 'SalePrice'],
      dtype='object')
```

Skewness: 1.88

Conclusion: The data is Positively Skewed (Right Skewed).



Skewness: 1.88

Conclusion: The data is Positively Skewed (Right Skewed).

#4 Upload Dataset

```
from google.colab import files  
uploaded = files.upload()
```

```
# Import Libraries  
import pandas as pd  
import matplotlib.pyplot as plt
```

```
# Read Dataset  
df = pd.read_csv("/content/Salary_dataset.csv")
```

```
# Display first 5 records  
print(df.head())
```

```
# Display column names  
print(df.columns)
```

```
# Box Plot
```



```
plt.figure(figsize=(7,5))
plt.boxplot(df["Salary"], patch_artist=True)
```

```
plt.title("Box Plot of Salary Data")
plt.ylabel("Salary")
plt.grid(True)
```

```
plt.show()
```

```
# Detect Outliers using IQR
Q1 = df["Salary"].quantile(0.25)
Q3 = df["Salary"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
```

```
outliers = df[(df["Salary"] < lower) | (df["Salary"] > upper)]
```

```
print("\nLower Limit:", lower)
print("Upper Limit:", upper)
print("\nNumber of Outliers:", len(outliers))
print("\nOutliers:")
print(outliers)
```

Saving Salary_dataset.csv to Salary_dataset (1).csv

```
Unnamed: 0  YearsExperience  Salary
```

```
0          0           1.2  39344.0
```

```
1          1           1.4  46206.0
```

```
2          2           1.6  37732.0
```

```
3          3           2.1  43526.0
```

```
4          4           2.3  39892.0
```

```
Index(['Unnamed: 0', 'YearsExperience', 'Salary'], dtype='object')
```



Lower Limit: -9014.25
Upper Limit: 166281.75

Number of Outliers: 0

Outliers:
Empty DataFrame
Columns: [Unnamed: 0, YearsExperience, Salary]
Index: []

#5 Upload Dataset

```
from google.colab import files  
uploaded = files.upload()
```

```
# Import Libraries  
import pandas as pd
```

```
# Read Dataset  
df = pd.read_csv("StudentsPerformance.csv")
```

```
# Display first 5 records  
print(df.head())
```

```
# Display column names
```

```

print("\nColumns:")
print(df.columns)

# Select Marks Column (Math Score)
marks = df["math score"]

# Calculate IQR
Q1 = marks.quantile(0.25)
Q3 = marks.quantile(0.75)
IQR = Q3 - Q1

# Calculate Lower and Upper Limits
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

print("\nLower Limit:", lower)
print("Upper Limit:", upper)

# Detect Outliers
outliers = df[(marks < lower) | (marks > upper)]

print("\nNumber of Outliers:", len(outliers))

print("\nOutliers:")
print(outliers[["math score"]])

```

OUTPUT:

Saving StudentsPerformance (1).csv to StudentsPerformance (1) (1).csv

	gender	race/ethnicity	parental level of education	lunch
0	female	group B	bachelor's degree	standard
1	female	group C	some college	standard
2	female	group B	master's degree	standard
3	male	group A	associate's degree	free/reduced
4	male	group C	some college	standard

	test preparation course	math score	reading score	writing score
0	none	72	72	74
1	completed	69	90	88
2	none	90	95	93
3	none	47	57	44
4	none	76	78	75

Columns:

```
Index(['gender', 'race/ethnicity', 'parental level of education', 'lunch',  
      'test preparation course', 'math score', 'reading score',  
      'writing score'],  
      dtype='object')
```

Lower Limit: 27.0

Upper Limit: 107.0

Number of Outliers: 8

Outliers:

	math score
17	18
59	0
145	22
338	24
466	26
787	19
842	23
980	8

#6 Upload Dataset

```
from google.colab import files  
uploaded = files.upload()
```

Import Libraries

```
import pandas as pd
```

Read Dataset

```
df = pd.read_csv("/content/Test-selected-columns.csv")
```

Display first 5 rows

```
print("First 5 Records:")  
print(df.head())
```

Check column names

```
print("\nColumns in Dataset:")  
print(df.columns)
```

Select Sales column

```
sales = df["Item_MRP"]
```

Calculate Q1, Q3 and IQR

```
Q1 = sales.quantile(0.25)
```

```
Q3 = sales.quantile(0.75)
```

```
IQR = Q3 - Q1
```

```

# Calculate lower and upper limits
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

print("\nLower Limit:", lower_limit)
print("Upper Limit:", upper_limit)

# Detect outliers
outliers = df[(sales < lower_limit) | (sales > upper_limit)]

print("\nNumber of Outliers:", len(outliers))

# Remove outliers
cleaned_df = df[(sales >= lower_limit) & (sales <= upper_limit)]

print("\nOriginal Dataset Shape:", df.shape)
print("Cleaned Dataset Shape:", cleaned_df.shape)

print("\nFirst 10 Rows of Cleaned Dataset:")
print(cleaned_df.head(10))

```

OUTPUT:

First 5 Records:

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type \
0	FDW58	20.750	Low Fat	0.007565	Snack Foods
1	FDW14	8.300	reg	0.038428	Dairy
2	NCN55	14.600	Low Fat	0.099575	Others
3	FDQ58	7.315	Low Fat	0.015388	Snack Foods
4	FDY38	NaN	Regular	0.118599	Dairy

	Item_MRP	Outlet_Identifier	Outlet_Establishment_Year	Outlet_Size \
0	107.8622	OUT049	1999	Medium
1	87.3198	OUT017	2007	NaN
2	241.7538	OUT010	1998	NaN
3	155.0340	OUT017	2007	NaN
4	234.2300	OUT027	1985	Medium

	Outlet_Location_Type
0	Tier 1
1	Tier 2
2	Tier 3
3	Tier 2
4	Tier 3

Columns in Dataset:

```
Index(['Item_Identifier', 'Item_Weight', 'Item_Fat_Content', 'Item_Visibility',  
      'Item_Type', 'Item_MRP', 'Outlet_Identifier',  
      'Outlet_Establishment_Year', 'Outlet_Size', 'Outlet_Location_Type'],  
      dtype='object')
```

Lower Limit: -43.009899999999999

Upper Limit: 323.44849999999997

Number of Outliers: 0

Original Dataset Shape: (5681, 10)

Cleaned Dataset Shape: (5681, 10)

First 10 Rows of Cleaned Dataset:

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility \
0	FDW58	20.750	Low Fat	0.007565
1	FDW14	8.300	reg	0.038428
2	NCN55	14.600	Low Fat	0.099575
3	FDQ58	7.315	Low Fat	0.015388
4	FDY38	NaN	Regular	0.118599
5	FDH56	9.800	Regular	0.063817
6	FDL48	19.350	Regular	0.082602
7	FDC48	NaN	Low Fat	0.015782
8	FDN33	6.305	Regular	0.123365
9	FDA36	5.985	Low Fat	0.005698

	Item_Type	Item_MRP	Outlet_Identifier \
0	Snack Foods	107.8622	OUT049
1	Dairy	87.3198	OUT017
2	Others	241.7538	OUT010
3	Snack Foods	155.0340	OUT017
4	Dairy	234.2300	OUT027
5	Fruits and Vegetables	117.1492	OUT046
6	Baking Goods	50.1034	OUT018
7	Baking Goods	81.0592	OUT027
8	Snack Foods	95.7436	OUT045
9	Baking Goods	186.8924	OUT017

	Outlet_Establishment_Year	Outlet_Size	Outlet_Location_Type
0	1999	Medium	Tier 1
1	2007	NaN	Tier 2
2	1998	NaN	Tier 3
3	2007	NaN	Tier 2
4	1985	Medium	Tier 3
5	1997	Small	Tier 1

6	2009	Medium	Tier 3
7	1985	Medium	Tier 3
8	2002	NaN	Tier 2
9	2007	NaN	Tier 2

#7 Upload Dataset

```
from google.colab import files
uploaded = files.upload()
```

```
# Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Read Dataset
df = pd.read_csv("/content/Test-selected-columns.csv")
```

```
# Display first 5 records
print("First 5 Records:")
print(df.head())
```

```
# Select Sales Column
sales = df["Item_MRP"]
```

```
# Calculate IQR
Q1 = sales.quantile(0.25)
Q3 = sales.quantile(0.75)
IQR = Q3 - Q1
```

```
# Lower and Upper Limits
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
```

```
# Remove Outliers
cleaned_df = df[(sales >= lower) & (sales <= upper)]
```

```
print("\nOriginal Dataset Shape:", df.shape)
print("Cleaned Dataset Shape:", cleaned_df.shape)
```

```
# Create Plots
plt.figure(figsize=(12,10))
```

```
# Histogram Before
plt.subplot(2,2,1)
plt.hist(df["Item_MRP"], bins=25,
```

```

        color="skyblue", edgecolor="black")
plt.title("Histogram Before Removing Outliers")
plt.xlabel("Sales")
plt.ylabel("Frequency")

# Box Plot Before
plt.subplot(2,2,2)
plt.boxplot(df["Item_MRP"], patch_artist=True)
plt.title("Box Plot Before Removing Outliers")
plt.ylabel("Sales")

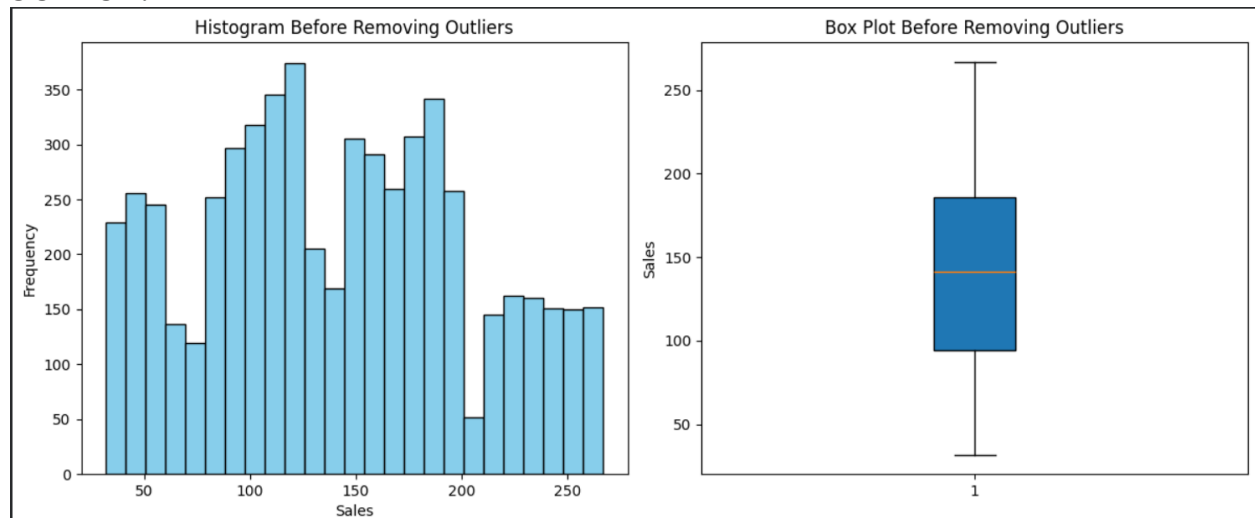
# Histogram After
plt.subplot(2,2,3)
plt.hist(cleaned_df["Item_MRP"], bins=25,
        color="lightgreen", edgecolor="black")
plt.title("Histogram After Removing Outliers")
plt.xlabel("Sales")
plt.ylabel("Frequency")

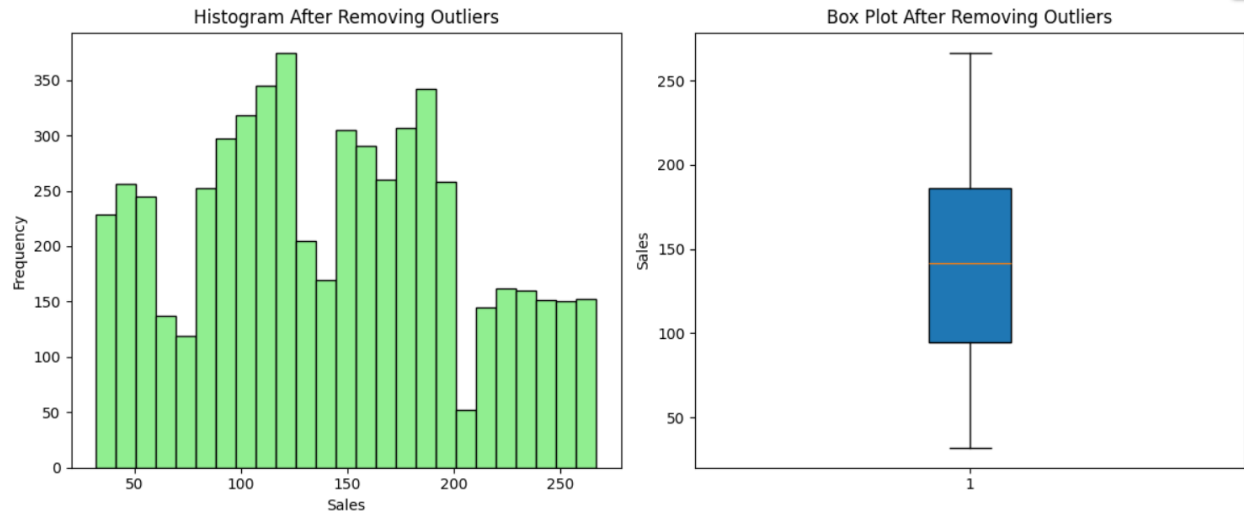
# Box Plot After
plt.subplot(2,2,4)
plt.boxplot(cleaned_df["Item_MRP"], patch_artist=True)
plt.title("Box Plot After Removing Outliers")
plt.ylabel("Sales")

plt.tight_layout()
plt.show()

```

OUTPUT:





#8

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("/content/gender_submission.csv")
```

```
# Dataset Information
```

```
print(df.info())
```

```
# Missing Values
```

```
print(df.isnull().sum())
```

```
# Descriptive Statistics
```

```
print(df.describe())
```

```
# Histograms
```

```
df.hist(figsize=(12,10))
```

```
plt.show()
```

```
# Box Plots
```

```
num_cols = df.select_dtypes(include="number").columns
```

```
for col in num_cols:
```

```
    plt.figure(figsize=(5,2))
```

```
    plt.boxplot(df[col].dropna(), vert=False)
```

```
    plt.title(col)
```

```
    plt.show()
```

```
# Detect Outliers
```

```
for col in num_cols:
```

```
    Q1 = df[col].quantile(0.25)
```

```
    Q3 = df[col].quantile(0.75)
```

```

IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[(df[col] < lower) | (df[col] > upper)]
print(col, ":", len(outliers))

# Remove Outliers
clean_df = df.copy()
for col in num_cols:
    Q1 = clean_df[col].quantile(0.25)
    Q3 = clean_df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    clean_df = clean_df[(clean_df[col] >= lower) & (clean_df[col] <= upper)]

# Save Cleaned Dataset
clean_df.to_csv("Titanic_Cleaned.csv", index=False)

print(clean_df.shape)

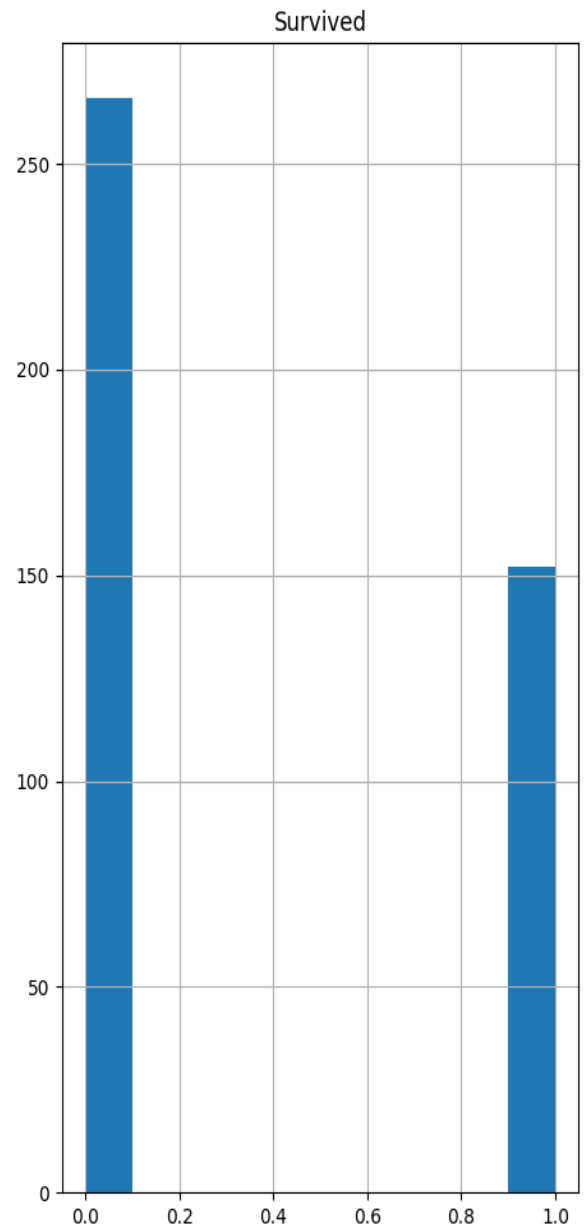
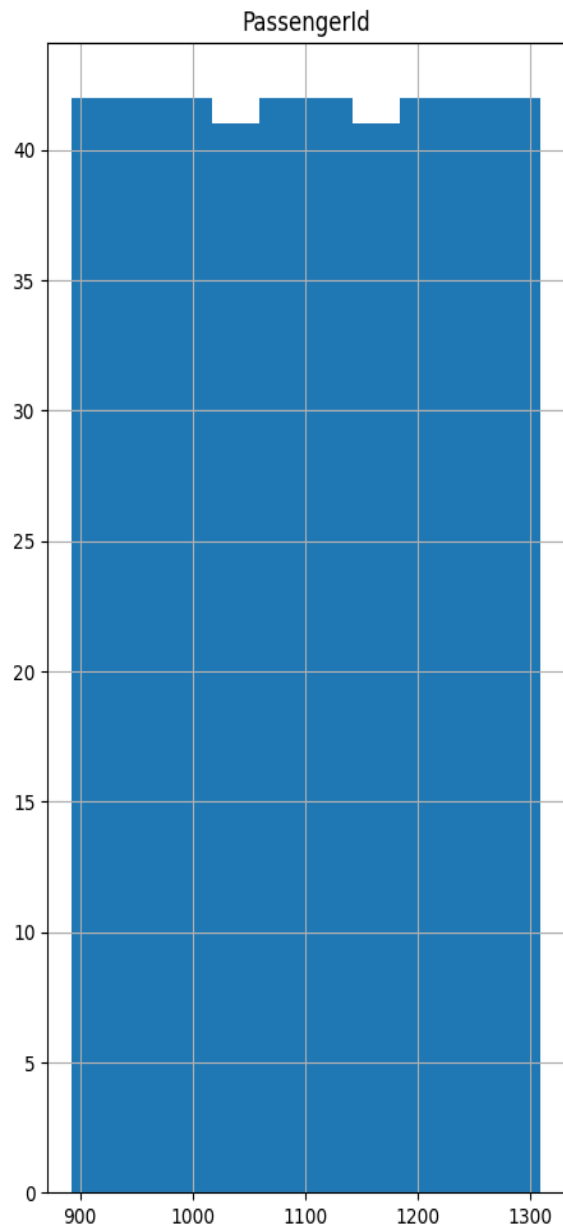
```

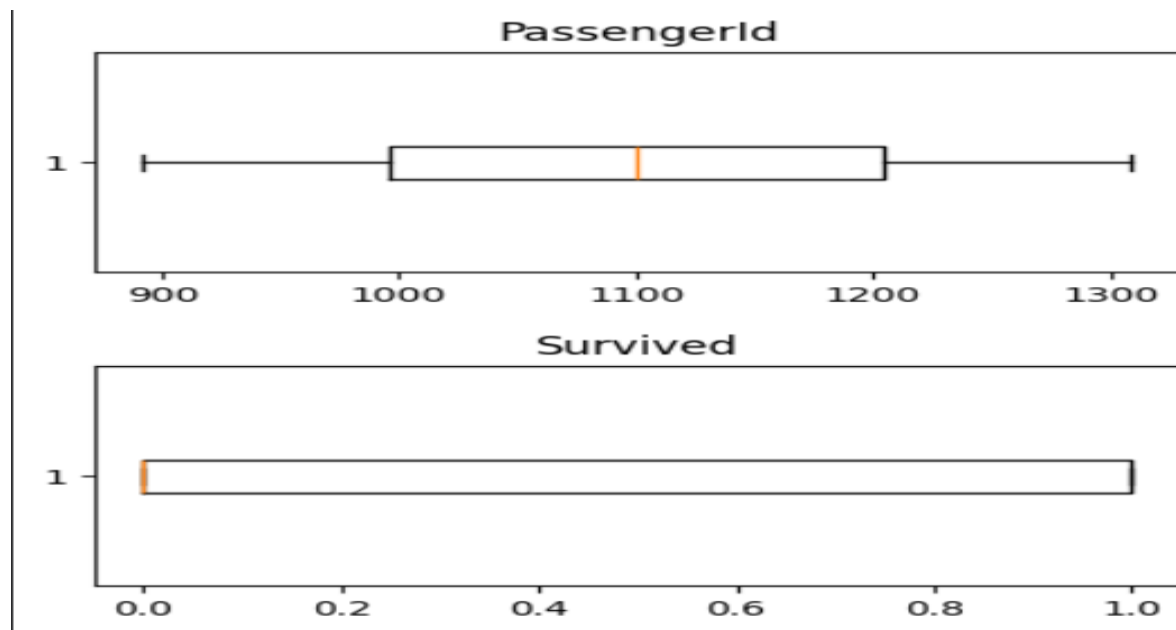
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  418 non-null    int64
1   Survived     418 non-null    int64
dtypes: int64(2)
memory usage: 6.7 KB
None
PassengerId    0
Survived       0
dtype: int64

```

	PassengerId	Survived
count	418.000000	418.000000
mean	1100.500000	0.363636
std	120.810458	0.481622
min	892.000000	0.000000
25%	996.250000	0.000000
50%	1100.500000	0.000000
75%	1204.750000	1.000000
max	1309.000000	1.000000





PassengerId : 0
Survived : 0
(418, 2)